

Contents

1	How to be a PLC specialist	3
1.1	PLC의 본질적 이해	4
1.2	현장 중심의 학습과 회사 적응력	5
1.3	PLC가 쉬운 이유 vs 어려운 이유	6
1.4	PLC 엔지니어의 기술적 스펙트럼	8
1.5	세계의 PLC 엔지니어 성장 루트	10
1.6	PLC 엔지니어링의 궁극의 경지	12
1.7	결론 — 시스템 철학으로의 진화	14

Chapter 1

How to be a PLC specialist

1.1 PLC의 본질적 이해

일단 plc라는 게 뭘까부터 생각해 보기를.

plc는 c나 c#처럼 그냥 하나의 언어일 뿐이라고 생각함.

하지만 실제로 공부를 하다 보면, 그게 단순한 언어라기보다 **제어의 수단**이라는 걸 깨닫게 됨.
즉, plc 자체보다 중요한 건 **plc 가지고 무엇을 제어할 것인가** 하는 질문임.

내가 장비를 제어할 것인가, 시설을 제어할 것인가.

이 고민이 선행되지 않으면 배움이 공허해짐.

plc를 독학으로 할 수 없는 이유가 바로 이 때문이라고 생각이 되는데,
제어라는 건 실제 물리적 장치와 전기 신호를 다루는 일이라서,
기구물·배선·모터·인버터 같은 걸 직접 만져보지 않으면 감이 안 옴.

물론 돈이 많아서 집에다가 서보 모터, 인버터 다 사서
튜닝도 해보고 파라미터도 넣어보고 기구물도 조립하고 배선도 하면 되지.
근데 뭐하러, 어차피 회사 가면 하루 종일 그걸 하게 되는데 ㅋㅋ

단순히 레더로 신호등 짜고 숫자 입력해서 출력하고,
그런 건 그냥 환경에 익숙해지는 수준일 뿐임.
회사에서 진짜 원하는 능력은 그런 게 아님.

결국 중요한 건, plc라는 도구를 통해 실제 시스템이 어떻게 돌아가는지를 이해하는 것.

요약하자면, plc는 단순한 프로그래밍 언어가 아니라
현실의 기계를 움직이게 하는 언어다.

무엇을 제어할 것인가를 먼저 생각하고,
그 시스템의 구조와 동작 원리를 이해하는 게 진짜 공부의 시작이다.

1.2 현장 중심의 학습과 회사 적응력

회사에서 신입에게 바라는 건, 코드를 잘 짜는 게 아님.

plc로 개발을 한다? 그건 대부분 신입이 할 일이 아님.

장비가 어떻게 돌아가는지도 모르는 사람이 개발을 맡는 건 불가능에 가까움.

회사에서 진짜 바라는 건 **적응력**이다.

그 회사가 장비업체건, 시설업체건 상관없이,

그 구조와 명칭을 제대로 알고, 동작의 원리를 이해하는 것.

그리고 **전장도면을 볼 줄 아는 능력**,

I/O 체크 정도는 스스로 할 수 있는 능력.

이게 신입으로서 가장 빨리 회사에 적응하는 방법이라고 생각함.

신입에게는 코딩보다 더 중요한 게 있음.

바로 현장과 장비를 이해하는 감각이다.

기계가 어떻게 움직이고, 센서가 어떤 신호를 주며,

그 신호가 PLC의 입력으로 들어와 어떤 출력을 만들게 되는지,

그 연결 고리를 머릿속에서 그릴 수 있어야 함.

회사에서는 이런 사람을 진짜로 신뢰함.

명령어 몇 개 아는 것보다, 실제로 공정의 흐름을 이해하고

문제가 생겼을 때 현장에서 바로 대응할 줄 아는 사람.

그게 현장형 엔지니어의 기본기다.

결국, plc를 잘 다루는 사람보다

시스템 전체를 읽을 줄 아는 사람이 먼저 살아남는다.

이게 진짜 현장 중심의 성장 방식이다.

1.3 PLC가 쉬운 이유 vs 어려운 이유

plc는 겉보기엔 쉽다. 하지만 실제로 깊이 들어가면 어렵다.

이 두 가지가 공존하는 이유를 정리해보면 다음과 같다.

쉽다 :

- 1) 일반적인 **BOOL 연산**만 알면 기본 명령어는 거의 다 다룰 수 있다.
- 2) 논리 구조가 직관적이라 방법만 알면 기본적인 연산은 금방 끄적거릴 수 있다.
즉, 회로도의 흐름을 이해하면 그대로 래더로 옮기기 쉽다는 뜻이다.

하지만 어렵다 :

- 1) 어떤 기능 하나를 제대로 구현하려면, 복붙이 아니라 직접 만들어야 한다.
소프트웨어라고 하지만 사실상 **하드웨어 수준의 디지털 신호 처리**에 가깝다.
설계 방식 자체가 FPGA와 유사하다.

일반 프로그래밍은 인터넷에 복사할 코드가 많고,

함수화나 모듈화가 쉬워서 내부 구조를 몰라도 된다.

하지만 PLC는 전부 알아야 한다. 한 줄 한 줄이 전기 신호니까.

- 2) 일반적인 자료 구조론을 적용하기가 어렵다.

예를 들어 Queue 자료형은 일반 언어에서는 객체의 메서드를 호출하면 끝이지만,

PLC에는 그런 명령어가 거의 없다. 심지어 아는 사람도 적고 자료도 없다.

Mitsubishi는 **Stack 명령어**는 지원하지만 Queue는 직접 만들어야 한다.

보통 **WSFL**을 이용해 간접적으로 구현하는 수밖에 없다.

- 3) 펌웨어만 알아도 되는 게 아니다. 반드시 **전기적인 지식**이 필요하다.

점점, 릴레이, 전원, 배선, 접지, 전류 용량 같은 기본기가 없으면 프로그램이 돌아가지 않는다.

- 4) 현장은 공장 설비 바로 옆이다. 즉, **실제 기계 옆에서 일하는 직업**이다.

코드를 짜면서 동시에 배선, 센서, 기구 움직임을 다 봐야 한다.

그래서 SI(시스템 통합) 쪽보다 훨씬 고된 경우가 많다.

5) 이런 환경에서 알고리즘도 짜고, 통신도 구현하고, 예외 처리도 완벽히 해야 한다.

하나라도 잘못되면 바로 **설비 정지나 대형사고**로 이어질 수 있다.

PLC 한 줄이 멈추면 공장이 멈추는 셈이다.

이런 이유로 국비로 진입하는 인력이 많아도,

실제로 깊이 다루는 사람은 여전히 한정적이다.

plc는 단순히 ‘배워서 쓰는 기술’이 아니라,

전기, 논리, 공정, 안전을 모두 통합적으로 이해해야 다룰 수 있는 영역이다.

1.4 PLC 엔지니어의 기술적 스펙트럼

plc 프로그래밍은 일반적인 절차형 프로그래밍과 다르다.

릴레이 래더 로직(RLL)은 원시적인 언어다.

대부분의 프로그래머는 서브루틴조차 쓰지 않는다.

그래서 소프트웨어 공학이 발전해온 방향과 다르게,

plc 세계는 여전히 “시간과 공학이 멈춘 영역”처럼 느껴진다.

그렇기 때문에 오히려 기초적인 **소프트웨어 설계 사고**를 적용하는 것만으로도 큰 차이를 만든다.

예를 들어, 코드 블록 간의 인터페이스를 명확히 정의하거나,

논리 구조를 기능별로 구분하는 것만으로 유지보수성이 급격히 올라간다.

plc 프로그래밍의 본질은 **불대수(Boolean Logic)**다.

plc를 잘하려면 불대수, 특히 **드모르간 법칙**을 반드시 익혀야 한다.

왜냐하면 대부분의 plc는 NOT 연산자를 직접 제공하지 않기 때문이다.

plc는 실시간 제어와 피드백의 언어다.

입력-논리-출력의 루프가 끊임없이 반복되는 시스템이며,

결국 이 구조가 제어의 본질이다.

plc 코드는 출력을 구동하는 논리이고,

그 논리가 구동하는 기계 시스템이 다시 입력을 만들어낸다.

이 둘이 서로를 구동하는 **피드백 회로**를 이룬다.

많은 plc 제어는 결국 **상태 전이(State Transition)** 문제다.

입력 변화가 감지되면 상태가 전이되고,

따라서 **유한 상태 기계(FSM)** 개념을 이해하는 것이 필수다.

plc는 과거 사건을 기억해야 한다.

그래서 많은 프로그램이 Set/Reset, Timer, Counter 구조로 이루어진다.

표면적으로는 논리적이지만, 실제로는 수많은 **사이드 이펙트**를 가진다.

대부분의 plc 프로그램은 “설비가 정상 동작한다”는 가정 하에 짜여 있다.

하지만 현실의 설비는 언젠가 반드시 고장 난다.

따라서 코드 안에는 **진단(Diagnostic) 로직**이 꼭 포함되어야 한다.

결국 이 모든 스펙트럼이 합쳐져야 진짜 **시스템 제어 엔지니어(System Control Engineer)**가 된다.

plc는 단순히 명령을 내리는 기계가 아니라,

전기 · 논리 · 시간 · 데이터가 만나는 **통합적 언어**다.

1.5 세계의 PLC 엔지니어 성장 루트

plc 엔지니어가 성장하는 방식은 나라마다 다르다.

하지만 공통점이 있다. 모두가 **현장 경험 + 표준 교육 + 통합 설계 능력**을 중심에 둔다.

독일 / 오스트리아 / 스위스

Berufsschule(직업학교) + Ausbildung(도제훈련) → 전기·메카트로닉스 기초

Techniker / Meister 상급 과정 → Siemens TIA Portal, Profibus, Profinet 심화

Fachhochschule(응용과학대) → 실무와 학위를 병행

핵심: 표준(IEC 61131-3), 현장경험, 통합 능력.

북유럽 (핀란드, 스웨덴, 덴마크)

plc를 임베디드 제어의 한 분기로 본다.

마이크로컨트롤러, Python, LabVIEW, ROS와 함께 배운다.

제어 알고리즘 → PLC → PC-based Control → IoT 연동.

핵심: 실시간 제어와 시스템 통합.

북미 (미국 / 캐나다)

Automation Technician Diploma → PLC, Motor Control, AC Drive 실습.

Control System Integrator 입사 → 현장 경험.

ISA 자격증(CCST, CAP) 취득.

핵심: 트러블슈팅과 문서화 능력.

일본

메카트로닉스형 접근. 제어공학 + 전기회로 + 시퀀스제어 병행.

Mitsubishi, Omron, Keyence 중심 실습.

현장에서는 배선, I/O, 인터락 설계부터 시작.

중급 이후 Structured Text, Ethernet/IP, Servo 제어로 확장.

핵심: 기구·전장·프로그램의 통합적 감각.

결국 어디서 배우든 핵심은 같다.

현장 + 표준 + 통합 설계.

이 세 가지를 갖춘 사람이 진짜 엔지니어로 성장한다.

1.6 PLC 엔지니어링의 궁극의 경지

plc 엔지니어의 성장에는 단계가 있다.

단순히 코드를 잘 짜는 기술자의 수준을 넘어,

현실의 시스템을 통합적으로 설계하는 사고로 확장되는 여정이다.

1 Entry : Machine-level Control Engineer

한 대의 기계를 안정적으로 돌릴 수 있는 사람.

래더 로직, I/O 매핑, 센서, 모터, 타이머, 카운터 — 이런 걸 이해하고 유지보수할 수 있다.

이 단계에서는 기계를 **자신의 손끝으로 움직이는 감각**을 익히는 게 전부다.

2 Intermediate : Line / Process Control Engineer

여러 대의 기계를 하나의 공정으로 연결할 줄 아는 사람.

Modbus, Ethernet/IP, Profibus, SCADA, HMI를 다룬다.

기계 간의 데이터, 타이밍, 안전 인터락을 제어한다.

3 Advanced : System Control Architect

기계, 전기, 네트워크, 소프트웨어를 하나의 구조로 통합하는 사람.

IEC 61131-3, Structured Text, Safety PLC, SCADA, MES, OPC UA, MQTT.

코드가 아니라 시스템의 구조를 설계하는 단계다.

4 Expert : Cyber-Physical System Engineer

현실의 기계와 디지털 세계를 하나의 유기체로 보는 사람.

PLC는 **현실과 정보 세계를 연결하는 신경망**이다.

실시간 분산 제어, Digital Twin, AI 제어, Cloud-Edge 연동.

시스템이 스스로 움직이게 만드는 사람.

5 Ultimate : System Philosopher

이 단계에선 제어의 대상이 바뀐다.

기계가 아니라 **시간, 사람, 시스템, 환경**을 설계한다.

시스템의 질서, 정보 구조, 에너지의 흐름을 통찰한다.

plc는 단순한 장비가 아니다.

그건 인간이 만든 세계의 **리듬과 논리**를 다루는 도구다.

그는 하드웨어, 소프트웨어, 사람, 시간, 공간 —

이 다섯 가지를 **하나의 생명체처럼 움직이게** 만든다.

기계를 제어하는 사람에서, 시스템의 존재 방식을 설계하는 사람으로.

1.7 결론 — 시스템 철학으로의 진화

이제 나는 안다.

plc를 배운다는 건 단순히 명령어를 익히는 게 아니라,
세상이 움직이는 방식을 이해하려는 시도라는 걸.

plc는 결국 **세계의 신경계**다.

전류가 흐르고, 신호가 왕복하며,
그 안에서 인간의 의도와 기계의 반응이 하나의 리듬으로 묶인다.

우리는 그 리듬을 설계하는 사람이다.

그 리듬이 끊어지지 않도록,
기계와 사람과 데이터가 동시에 살아 움직이게 만드는 사람.

처음엔 단순히 모터를 돌리고,

센서를 읽고, 불을 켜고 끄는 일에서 시작했지만,

점점 그것이 **하나의 생명체처럼 연결된 세계**를 다루는 일이라는 걸 깨닫게 된다.

그래서 진짜 엔지니어는 기술자가 아니라 철학자다.

그는 제어 루프 속에서 인과의 흐름을 보고,
시스템의 안정성 속에서 존재의 질서를 본다.

plc는 그 철학을 실험하는 무대다.

기계, 전기, 코드, 인간 — 모든 것이 만나는 교차점.